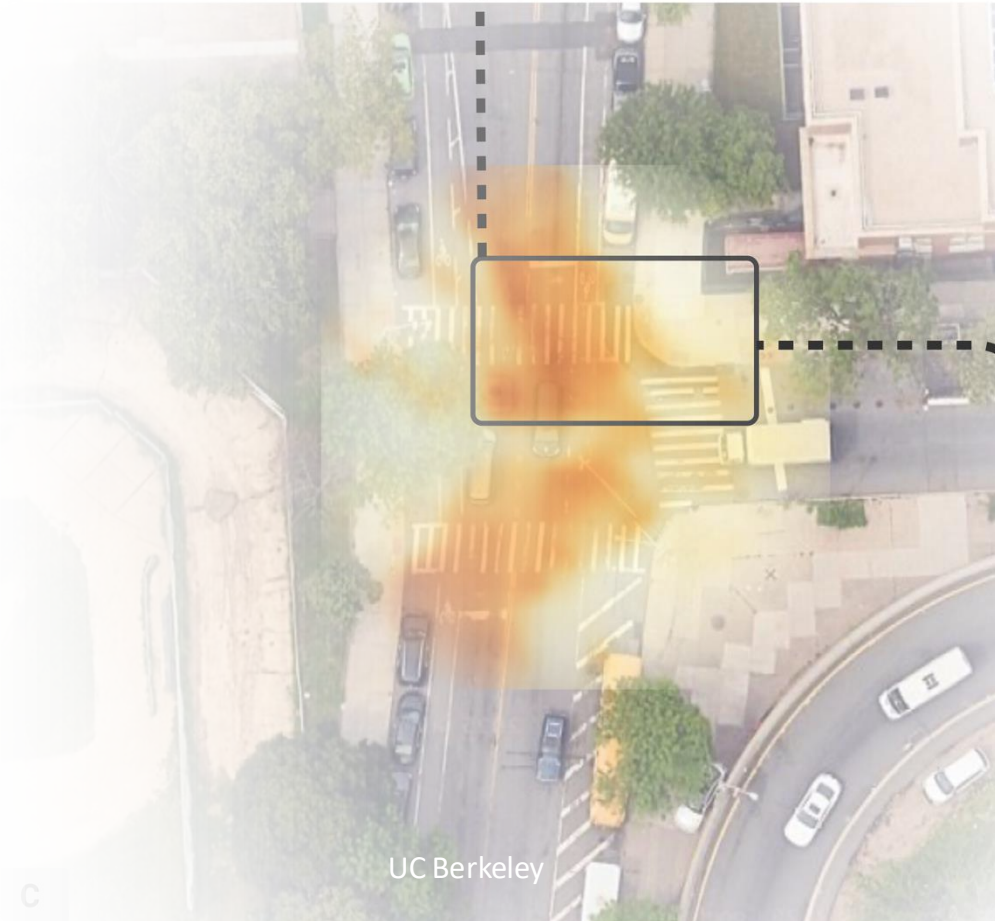
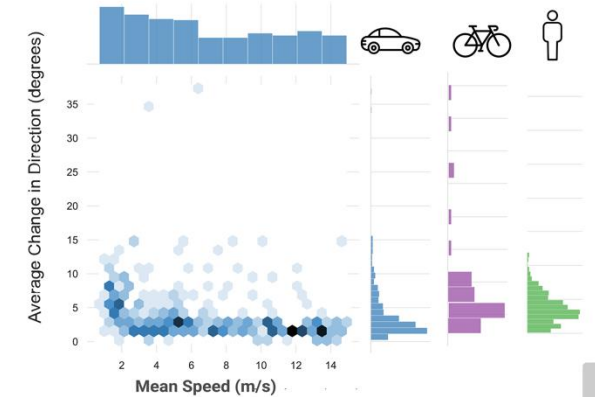
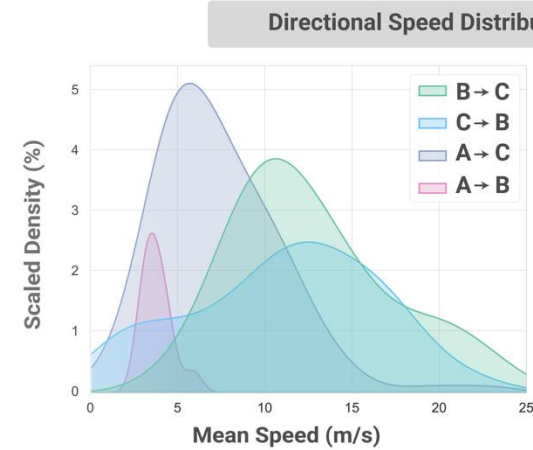
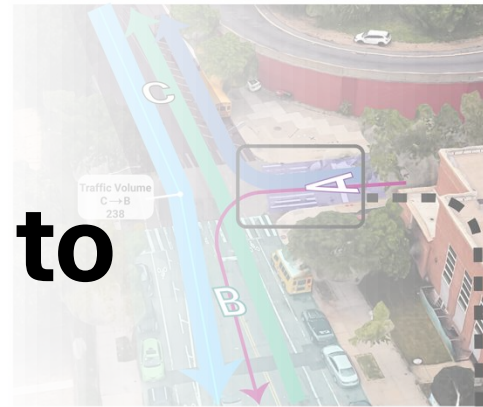


Introduction to Urban Data Analytics

Fall 2026

Maryam Hosseini

City & Regional Planning



UC Berkeley

B

D

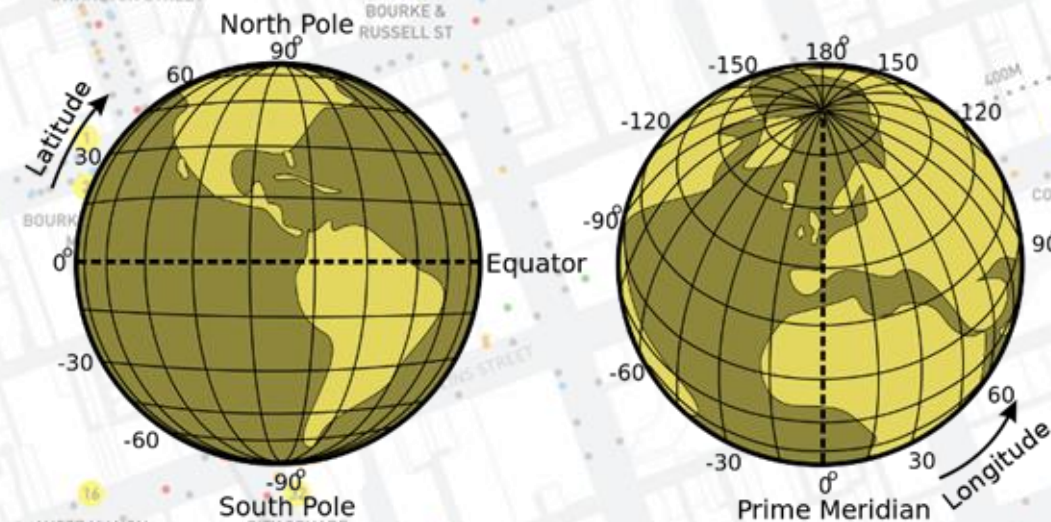
Geospatial Data



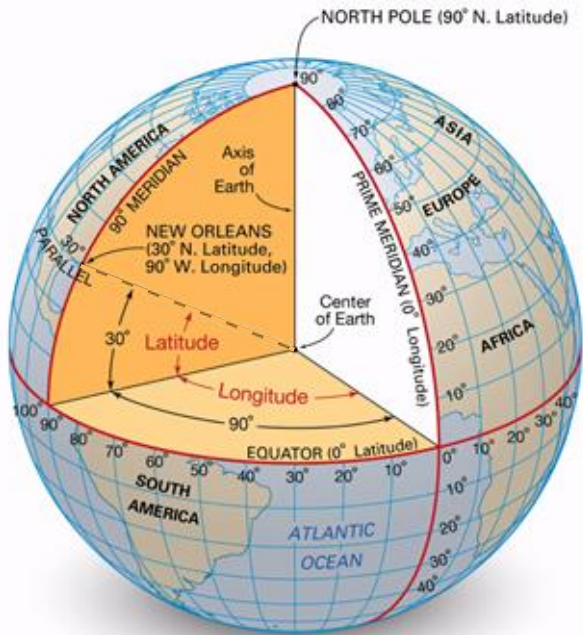
Projection, Vector & Raster Data

Geospatial Data

- Geospatial data is data with Earth's surface as the reference frame: **data tied to locations on Earth's surface**
- How can we uniquely identify any location on Earth?
- By using a geographic coordinate grid
- Any point on Earth can be specified with a pair of coordinates:
 - latitude (north–south)
 - longitude (east–west)

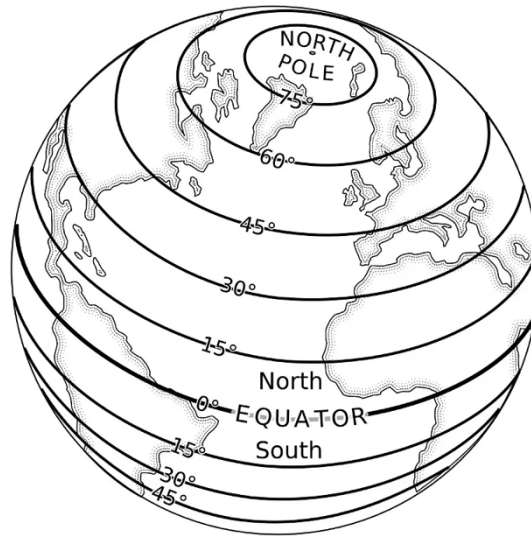


Latitude and Longitude



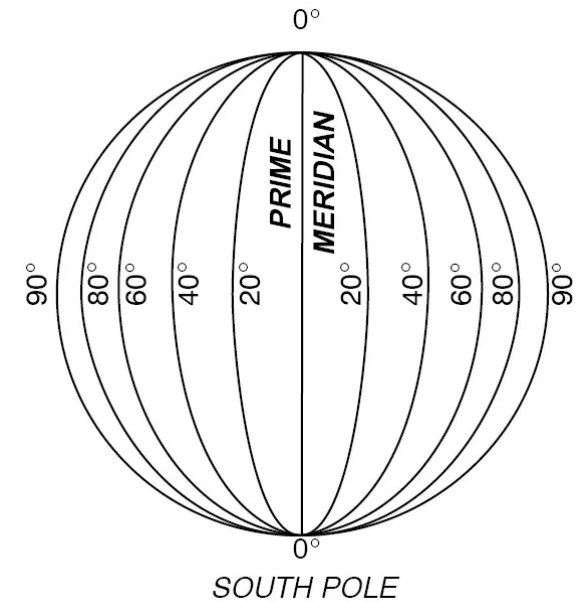
© Encyclopædia Britannica, Inc.

The origin (0,0), is where **Prime Meridian** and **Equator** intersect



Lines of Latitude

The lines that run east and west each have a constant latitude value and are called **parallels**.

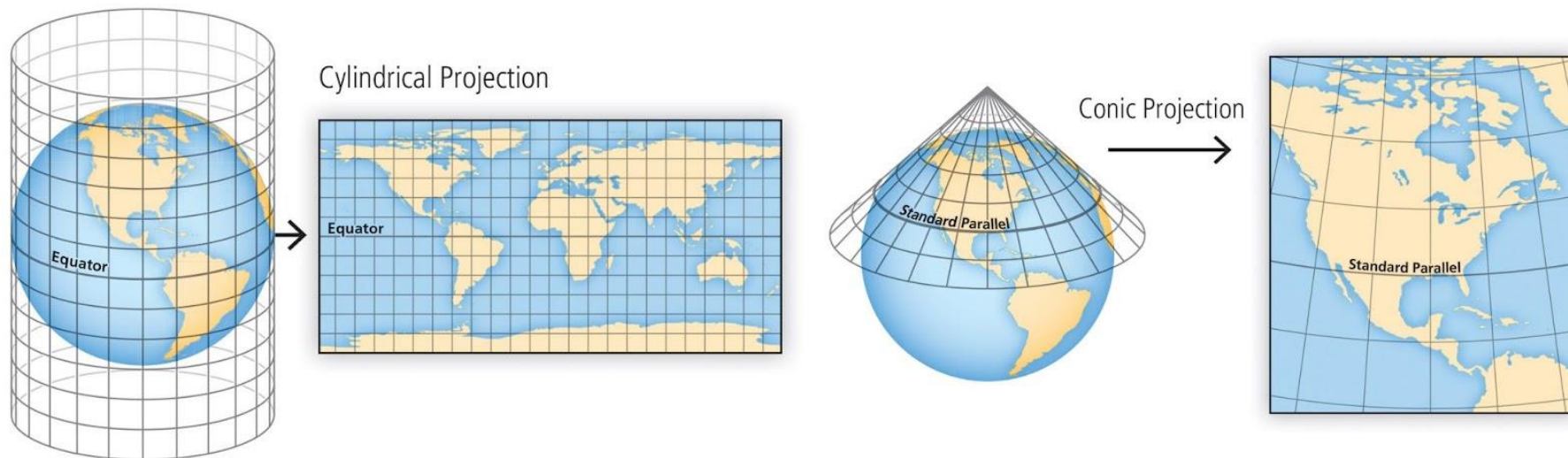


Lines of Longitude

The lines that run north and south each have a constant longitude value and are called **meridians**.

Projection

- **Projection:** mathematical transformations that convert the 3D, curved surface of the Earth into a 2D, flat map
- Projection is a key concept of cartography, the art and science of making maps



Coordinate Reference System (CRS)

A **coordinate reference system** (CRS) defines how the two-dimensional, projected map relates to real places on the earth.



(lon, lat)

Spherical Model
**Geographic
Coordinate System**
(GCS)



(x, y)

Flat Model
**Projected
Coordinate System**
(PCS)

Why bother with different CRS?

All projections distort some combination of:

- Shape
- Size
- Distance
- Direction
- Combination of these

The best projection depends on the use of the map.

World Mercator projection with country going to true size



@neilrkaye

Coordinate Reference System (CRS)



CRS connects data to the Earth's surface using a mathematical model

Geographic CRS:

- Uses latitude and longitude (angular degrees).
- Defines positions using latitude and longitude (angles) measured from the equator and prime meridian **on a specified datum/ellipsoid (e.g., WGS 84)**.
- Common example: **EPSG:4326 (WGS84)** (World Geodetic System (WGS) used in GPS).

Projected CRS

- Converts Earth's curved surface into a **2D map** with X, Y coordinates.
- Uses **meters or feet** instead of degrees
- Introduces **distortions**: different regions adopt their own CRS
- **Common example: EPSG:3857 (Web Mercator)** widely used for web mapping, **Significant area distortion near the poles.**

Geographic Information System (GIS)

“A system for
capturing, storing, checking, integrating,
manipulating, analyzing & displaying
spatial data”

(British Department of the Environment, 1987)

Where are we, what we are aiming for

- In this course, we move from **question** → **analysis** → **evidence** → **answer**.
- Start with a question, learn the techniques, and use them to **attempt an answer**.
- Proprietary tools (e.g., ArcGIS Pro) can be powerful, but access is often limited by licensing and cost.
- Open-source GIS tools (e.g., QGIS) are strong alternatives, but some workflows become slow, repetitive, or difficult to scale through the UI alone.
- We are learning Python not just to “code,” but to build **repeatable, scalable analysis workflows**.

What is expected?

- You are not expected to memorize every syntax detail we have covered. Forgetting syntax is normal.
- You should remember what is possible and how to ask the right question: “What was the code to **do X**?” At this stage, **X (the task)** matters more than memorizing the exact syntax.
- With enough practice, the syntax will start to stick, it needs to be **typed** so you build a muscle memory.

Geospatial Data Analysis

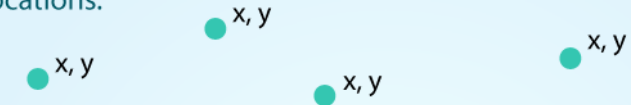
- Read data → input to our algorithm
 - Clean data → the process in cleaning alg.
 - Geometric or tabular transformation/operations
 - Filter / slice
 - Aggregate
 - Change the data by applying some transformations
 - Describe their relationship (predicates)
 - Visual exploration and visual analytics
- Write data → output of some process/alg.

Vector Data

- Vectors consist of vertices (x, y) that define spatial shapes.
- Their arrangement determines the type:
 - **Points:** Single x, y coordinate (e.g., POI sites)
 - **Lines:** Two or more connected vertices forming a continuous line (e.g., roads)
 - **Polygons:** Three or more vertices that are connected and closed (e.g., boundaries)

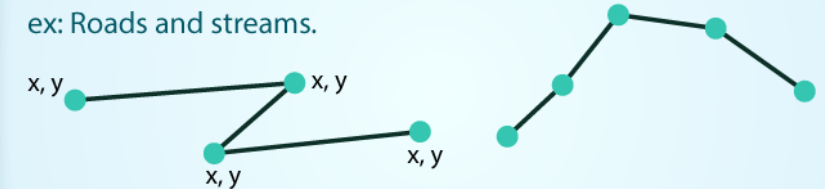
POINTS: Individual **x, y** locations.

ex: Center point of plot locations, tower locations, sampling locations.



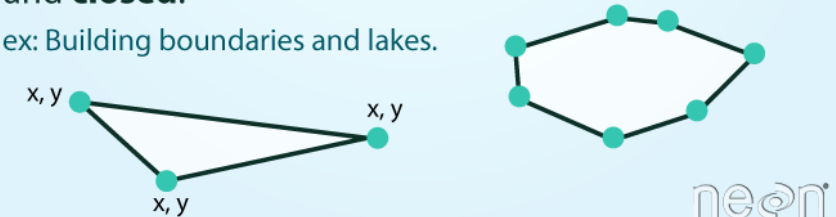
LINES: Composed of many (at least 2) vertices, or points, that are connected.

ex: Roads and streams.



POLYGONS: 3 or more vertices that are connected and **closed**.

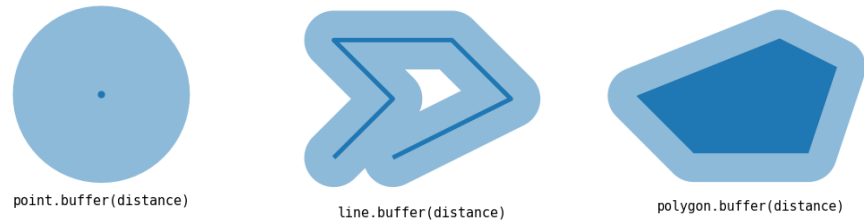
ex: Building boundaries and lakes.



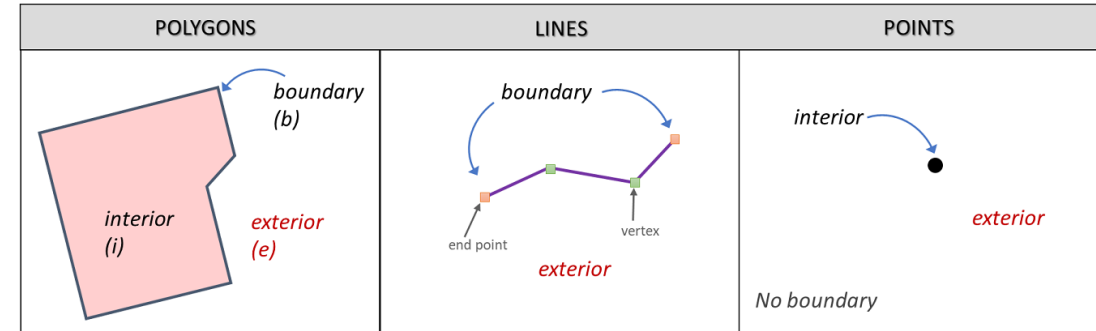
neon

Image Source: National Ecological Observatory Network (NEON)

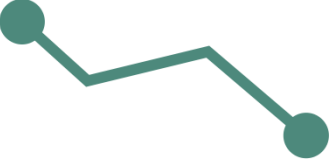
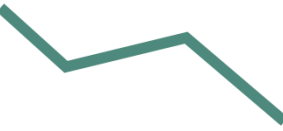
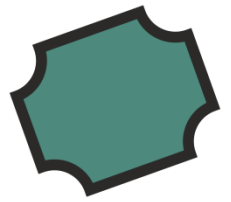
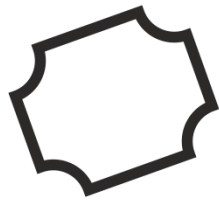
Geometric Manipulation



- **Point buffered** → Polygon
- **Line buffered** → Polygon
- **Polygon buffered** → Polygon



			
	Polygon	Line	Point
Area	True	False	False
Length	True	True	False

Data type	Dim.	Whole shape	Interior	Boundary	Exterior
Point	0				
LineString	1				
LinearRing	1				
Polygon	2				

<https://pythongis.org/>

Geopandas



Geospatial analysis in Python

Geopandas

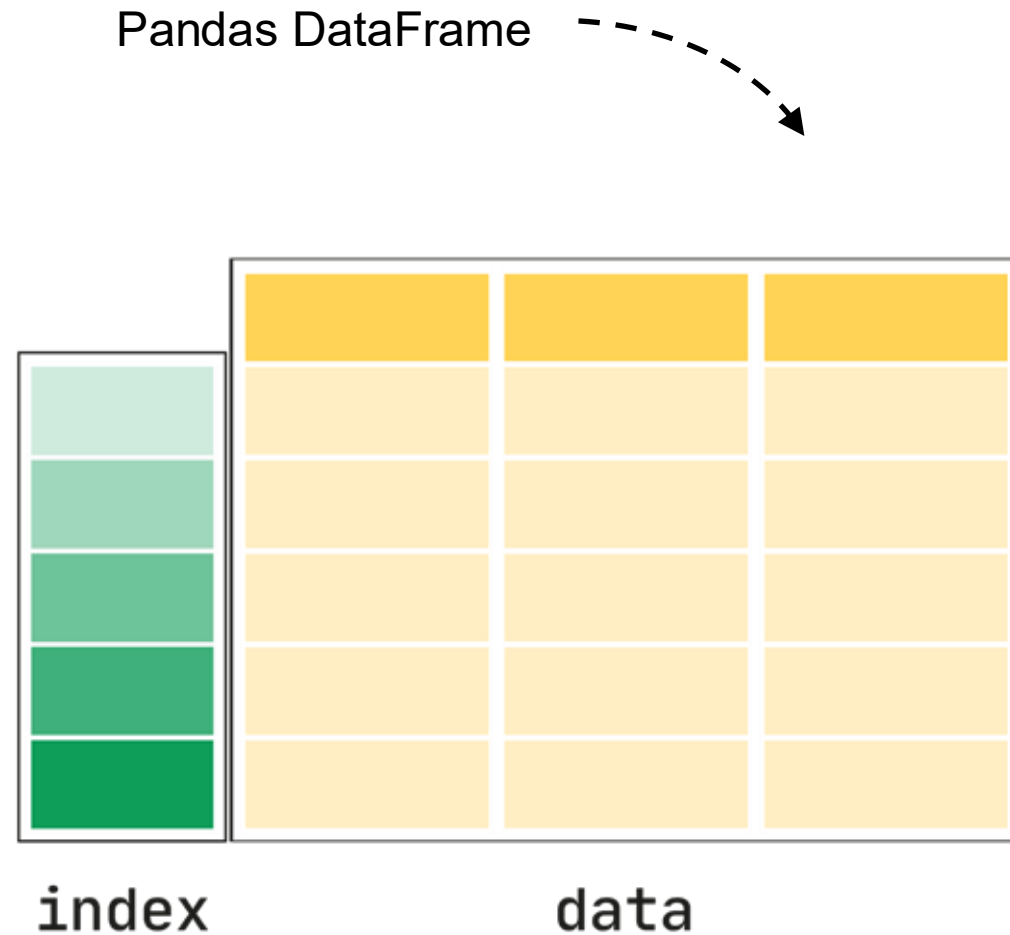
- A Python package that handles geospatial data:

```
import geopandas as gpd
```

- GeoPandas, extends **pandas** by adding support for geospatial data
- The core data structures in GeoPandas, similar to pandas, is GeoDataFrame that can store geometry columns and perform spatial operations
- The gpd.GeoSeries, a subclass of pandas.Series, handles geometries

GeoDataFrame

- GeoDataFrame is a combination of `pd.Series` with tabular data, and `gpd.GeoSeries`, with geometries (points, polygons, etc.)

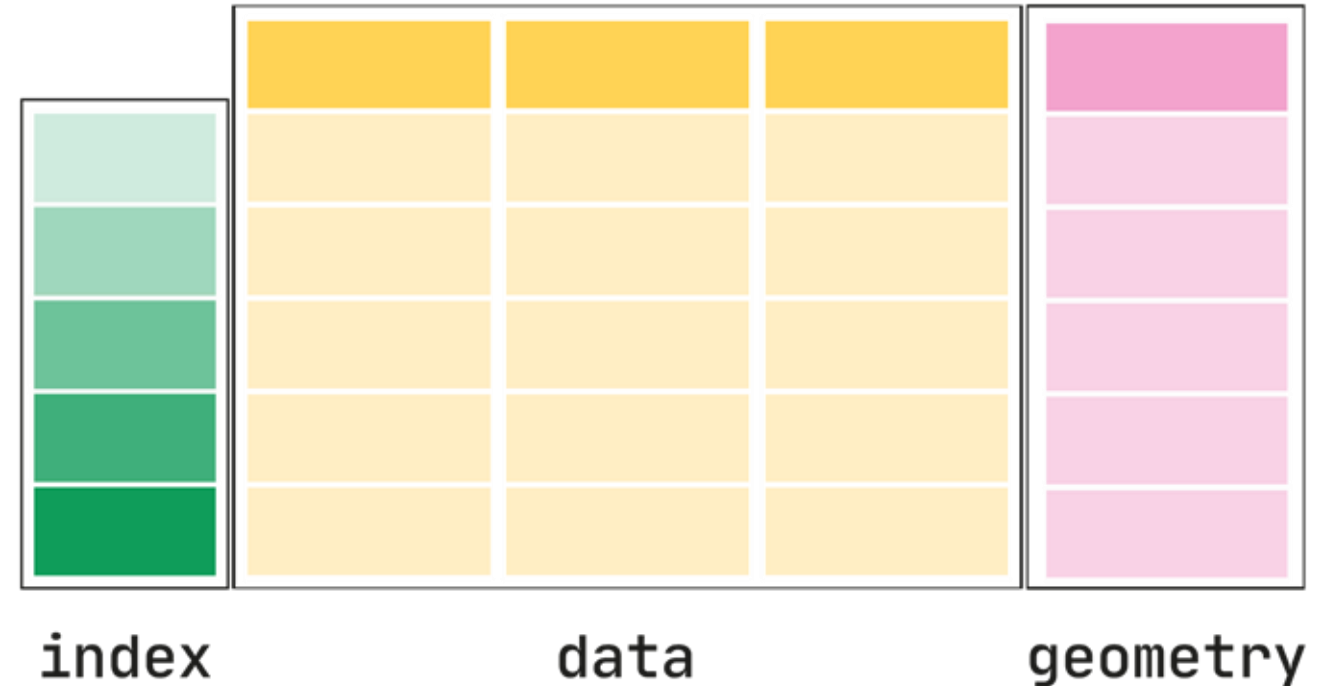
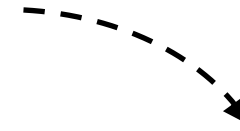


GeoPandas

Dependencies

- Geometry handling functionality `shapely`
- Geo-spatial file format support of `pyogrio`
- Map projection `pyproj`
- `GDAL`

GeoPandas GeoDataFrame

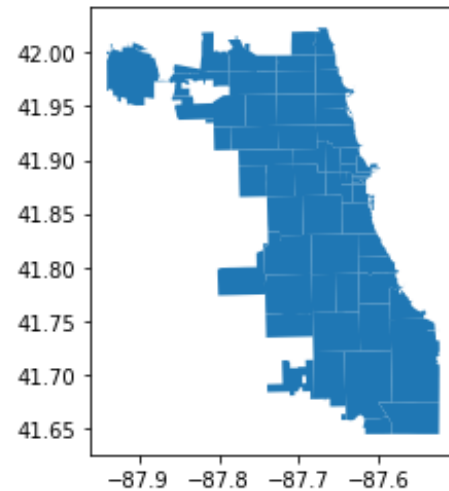


Reading files

```
1 gdf = gpd.read_file('chicago.geojson')
```

```
1 gdf.plot()
```

<AxesSubplot:>



```
1 gdf
```

	objectid	shape_area	shape_len	zip	geometry
0	33	106052287.488	42720.0444058	60647	MULTIPOLYGON (((-87.67762 41.91776, -87.67761 ...
1	34	127476050.762	48103.7827213	60639	MULTIPOLYGON (((-87.72683 41.92265, -87.72693 ...
2	35	45069038.4783	27288.6096123	60707	MULTIPOLYGON (((-87.78500 41.90915, -87.78531 ...
3	36	70853834.3797	42527.9896789	60622	MULTIPOLYGON (((-87.66707 41.88885, -87.66707 ...
4	37	99039621.2518	47970.1401531	60651	MULTIPOLYGON (((-87.70656 41.89555, -87.70672 ...
...	...	...	...	...	...
56	57	155285532.005	53406.9156168	60623	MULTIPOLYGON (((-87.69479 41.83008, -87.69486 ...
57	58	211114779.439	58701.3253749	60629	MULTIPOLYGON (((-87.68306 41.75786, -87.68306 ...
58	59	211696050.967	58466.1602979	60620	MULTIPOLYGON (((-87.62373 41.72167, -87.62388 ...
59	60	125424284.172	52377.8545408	60637	MULTIPOLYGON (((-87.57691 41.79511, -87.57700 ...
60	61	167872012.644	53040.9070778	60619	MULTIPOLYGON (((-87.58592 41.75150, -87.58592 ...

61 rows × 5 columns

Projection

```
1 gdf
```

	objectid	shape_area	shape_len	zip	geometry
0	33	106052287.488	42720.0444058	60647	MULTIPOLYGON (((-87.67762 41.91776, -87.67761 ...
1	34	127476050.762	48103.7827213	60639	MULTIPOLYGON (((-87.72683 41.92265, -87.72693 ...
2	35	45069038.4783	27288.6096123	60707	MULTIPOLYGON (((-87.78500 41.90915, -87.78531 ...
3	36	70853834.3797	42527.9896789	60622	MULTIPOLYGON (((-87.66707 41.88885, -87.66707 ...
4	37	99039621.2518	47970.1401531	60651	MULTIPOLYGON (((-87.70656 41.89555, -87.70672 ...
...	...	...	...	...	...
56	57	155285532.005	53406.9156168	60623	MULTIPOLYGON (((-87.69479 41.83008, -87.69486 ...
57	58	211114779.439	58701.3253749	60629	MULTIPOLYGON (((-87.68306 41.75786, -87.68306 ...
58	59	211696050.967	58466.1602979	60620	MULTIPOLYGON (((-87.62373 41.72167, -87.62388 ...
59	60	125424284.172	52377.8545408	60637	MULTIPOLYGON (((-87.57691 41.79511, -87.57700 ...
60	61	167872012.644	53040.9070778	60619	MULTIPOLYGON (((-87.58592 41.75150, -87.58592 ...

61 rows × 5 columns

Projecting to EPSG:3395

```
1 gdf = gdf.to_crs("EPSG:3395")
```

```
1 gdf
```

	objectid	shape_area	shape_len	zip	geometry
0	33	106052287.488	42720.0444058	60647	MULTIPOLYGON (((-9760228.181 5120114.708, -976...
1	34	127476050.762	48103.7827213	60639	MULTIPOLYGON (((-9765706.326 5120843.341, -976...
2	35	45069038.4783	27288.6096123	60707	MULTIPOLYGON (((-9772181.764 5118831.519, -977...
3	36	70853834.3797	42527.9896789	60622	MULTIPOLYGON (((-9759053.446 5115807.386, -975...
4	37	99039621.2518	47970.1401531	60651	MULTIPOLYGON (((-9763449.188 5116805.817, -976...
...	...	...	...	...	...
56	57	155285532.005	53406.9156168	60623	MULTIPOLYGON (((-9762139.685 5107055.000, -976...
57	58	211114779.439	58701.3253749	60629	MULTIPOLYGON (((-9760833.554 5096312.536, -976...
58	59	211696050.967	58466.1602979	60620	MULTIPOLYGON (((-9754228.915 5090933.835, -975...
59	60	125424284.172	52377.8545408	60637	MULTIPOLYGON (((-9749017.532 5101851.550, -974...
60	61	167872012.644	53040.9070778	60619	MULTIPOLYGON (((-9750019.820 5095367.709, -975...

61 rows × 5 columns

Projection

```
1 gdf
```

objectid		<Geographic 2D CRS: EPSG:4326> gdf.crs			geometry
0	33	1	Name: WGS 84		-87.67761 ...
1	34	1	Axis Info [ellipsoidal]:		-87.72693 ...
2	35	4	- Lat[north]: Geodetic latitude (degree)		-87.78531 ...
3	36	7	- Lon[east]: Geodetic longitude (degree)		-87.66707 ...
4	37	9	Area of Use:		-87.70672 ...
...	...	...	- name: World.		...
56	57	1	- bounds: (-180.0, -90.0, 180.0, 90.0)		-87.69486 ...
57	58	1	Datum: World Geodetic System 1984 ensemble		-87.68306 ...
58	59	1	- Ellipsoid: WGS 84		...
59	60	1	- Prime Meridian: Greenwich		...
60	61	1	MULTIPOLYGON (((-87.62373 41.72167, -87.62388 ...		...

61 rows × 5 columns

Projecting to EPSG:3395

```
1 gdf = gdf.to_crs("EPSG:3395")
```

```
1 gdf
```

objectid		shape	<Derived Projected CRS: EPSG:3395> gdf.crs		geometry
0	33	1060522	Name: WGS 84 / World Mercator		976...
1	34	1274760	Axis Info [cartesian]:		976...
2	35	4506903	- E[east]: Easting (metre)		977...
3	36	7085383	- N[north]: Northing (metre)		975...
4	37	9903962	Area of Use:		976...
...	...	...	- name: World between 80°S and 84°N.		...
56	57	1552855	- bounds: (-180.0, -80.0, 180.0, 84.0)		976...
57	58	2111147	Coordinate Operation:		976...
58	59	2116960	- name: World Mercator		975...
59	60	1254242	- method: Mercator (variant A)		974...
60	61	1678720	Datum: World Geodetic System 1984 ensemble		975...
...	...	...	- Ellipsoid: WGS 84		...
...	...	...	- Prime Meridian: Greenwich		...

61 rows × 5 columns

GeoPandas: Geocoding

```
gpd.tools.geocode("address")
```

```
region1 = "Boston Common, Boston, MA"  
gdf = gpd.tools.geocode(region1)  
gdf
```

	geometry	address
0	POINT (-71.06607 42.35504)	Boston Common, Boston, Massachusetts, United S...

Geocode a List of Addresses

```
addresses = [  
    "1600 Amphitheatre Parkway, Mountain View, CA",  
    "1 Apple Park Way, Cupertino, CA",  
    "350 5th Ave, New York, NY",  
    "221B Baker Street, London, UK",  
    "Bauer Wurster Hall, Berkeley, CA",  
    "Golden Gate Bridge, San Francisco, CA",  
    "1 Ferry Building, San Francisco, CA",  
    "Fantastic Comics, 2026 Shattuck Ave, Berkeley, CA"  
]  
  
# Geocode addresses using OpenStreetMap's Nominatim  
gdf = gpd.tools.geocode(addresses)  
gdf
```

	geometry	address
0	POINT (-122.08461 37.42176)	Google Headquarters, 1600, Amphitheatre Parkwa...
1	POINT (-122.01139 37.33484)	Apple Park, 1, Apple Park Way, 94087, Apple Pa...
2	POINT (-73.98566 40.74844)	Empire State Building, 350, 5th Avenue, 10118,...
3	POINT (-0.15847 51.52376)	The Sherlock Holmes Museum and shop, 221b, Bak...
4	POINT (-122.2549 37.87058)	Bauer Wurster Hall, Ong & Ong Plaza, 94720, On...
5	POINT (-122.47582 37.80766)	Golden Gate Bridge, 94129, San Francisco, Cali...
6	POINT (-122.39347 37.79555)	San Francisco Ferry Building, 1, Ferry Plaza, ...
7	POINT (-122.26883 37.87142)	2026, Shattuck Avenue, 94704, Shattuck Avenue,...

Simple operations

Compute & plot polygon area

```
1 gdf.area
```

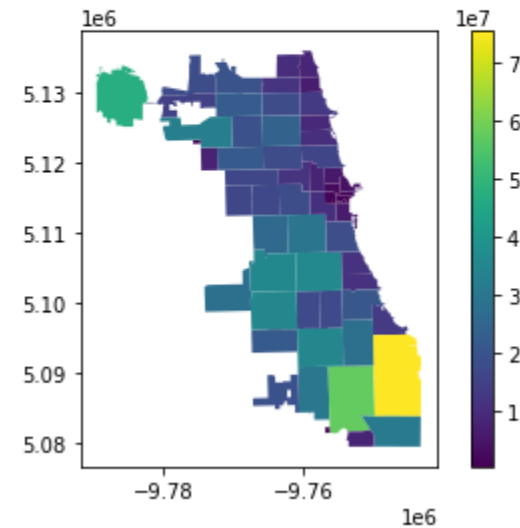
0	1.774279e+07
1	2.132685e+07
2	7.540024e+06
3	1.184732e+07
4	1.656003e+07
	...
56	2.592093e+07
57	3.515998e+07
58	3.521726e+07
59	2.089192e+07
60	2.793029e+07

Length: 61, dtype: float64

```
1 gdf['area'] = gdf.area
```

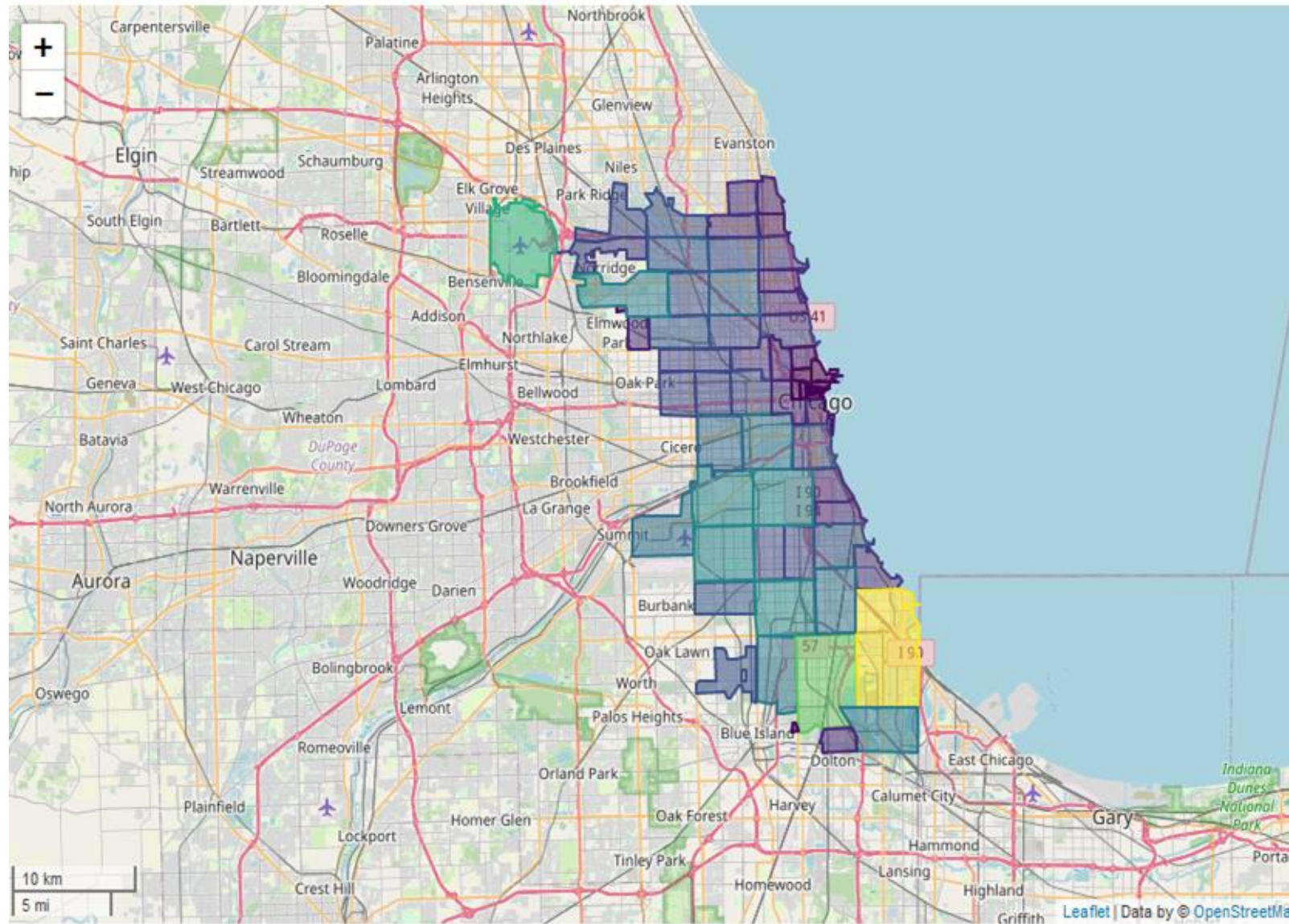
```
1 gdf.plot("area", legend=True)
```

<AxesSubplot:>



Interactive Map

```
1 gdf.explore("area", legend=False)
```



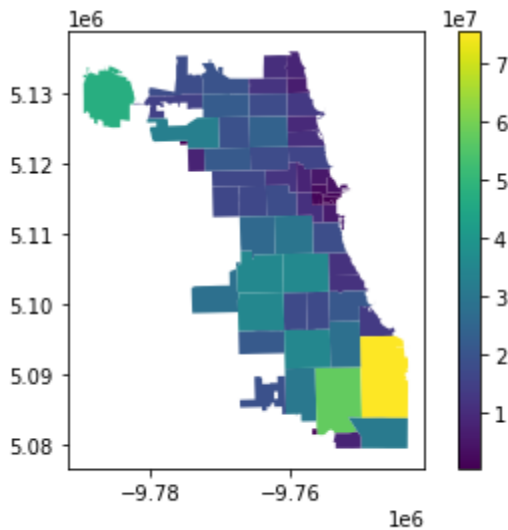
Polygon to Point

Changing geometry

```
1 gdf['area'] = gdf.area
```

```
1 gdf.plot("area", legend=True)
```

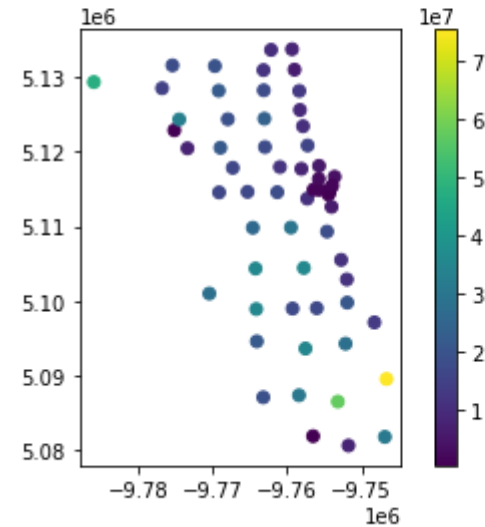
<AxesSubplot:>



```
1 gdf['centroid'] = gdf.centroid
```

```
1 gdf = gdf.set_geometry('centroid')  
2 gdf.plot("area", legend=True)
```

<AxesSubplot:>



Geometry operations: Buffer

```
1 gdf["buffered"] = gdf.buffer(1000)
```

→ Creating buffer around zip areas

```
2 gdf["buffered_centroid"] = gdf["centroid"].buffer(2000)
```

→ Creating buffer around centroids

```
1 ax = gdf["buffered"].plot(alpha=.5)
```

```
2 gdf["buffered_centroid"].plot(ax=ax, color="red", alpha=.5)
```

```
3 gdf["boundary"].plot(ax=ax, color="white", linewidth=.5)
```

<AxesSubplot:>

